

COMM114: MLE and EM

Guoqiang Zhang

September 2025

Maximum Likelihood Estimation (MLE)

Definition

Maximum likelihood estimation (MLE) is a method of estimating the parameters of an assumed probability distribution, given some observed data. This is achieved by maximizing a likelihood function so that, under the assumed statistical model, the observed data is most probable. (from wiki)

Problem Formulation:

- Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
- Select a parametric probability model $p_{\theta}(\cdot)$
- Fitting the data by maximizing the following likelihood function

$$\begin{aligned}\theta_{MLE}^* &= \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(\mathbf{x}_i) && \text{[product of likelihood]} \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i) && \text{[summation of log likelihood]}\end{aligned}$$

MLE for Fitting Scalar Gaussian distribution

- Input data set $\{x_i \in \mathbb{R} | i = 1, 2, \dots, n\}$
- Let $p_{\theta}(\cdot)$ be the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ where $\theta = \{\mu, \sigma\}$.
- Fitting the data by maximizing

$$\begin{aligned}\mu^*, \sigma^* &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log p_{\theta}(x_i) \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log \frac{1}{\sigma \sqrt{2\pi}} e^{-(x_i - \mu)^2 / 2\sigma^2} \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \left(- (x_i - \mu)^2 / 2\sigma^2 - \log \sigma \right) \\ &= \arg \min_{\theta} \underbrace{\frac{1}{n} \sum_{i=1}^n \left((x_i - \mu)^2 / 2\sigma^2 + \log \sigma \right)}_{L(\mu, \sigma)}\end{aligned}$$

MLE for Fitting Scalar Gaussian distribution (cont.)

- Set the derivative of $L(\mu, \sigma)$ with respect to μ and σ to be zero

$$\frac{\partial L(\mu, \sigma)}{\partial \mu} = 0 \quad (1)$$

$$\frac{\partial L(\mu, \sigma)}{\partial \sigma} = 0. \quad (2)$$

- Solving the above two equations in terms of μ and σ produces

$$\mu^* = \frac{1}{n} \sum_{i=1}^n x_i$$
$$\sigma^* = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu^*)^2}$$

Exercise

Computing μ^* by solving $\frac{\partial L(\mu, \sigma)}{\partial \mu} = 0$:

$$\frac{\partial L(\mu, \sigma)}{\partial \mu} = 0$$

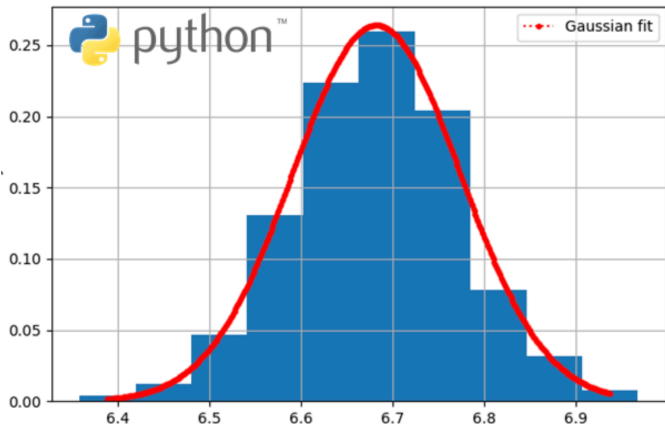
$$\Rightarrow \frac{\partial}{\partial \mu} \frac{1}{n} \sum_{i=1}^n \left((x_i - \mu)^2 / 2\sigma^2 + \log \sigma \right) = 0$$

$$\stackrel{(a)}{\Rightarrow} \frac{1}{n} \sum_{i=1}^n \left((x_i - \mu) / \sigma^2 \right) = 0$$

$$\stackrel{(b)}{\Rightarrow} \frac{1}{n} \sum_{i=1}^n \left((x_i - \mu) \right) = 0$$

$$\stackrel{(c)}{\Rightarrow} \mu^* = \frac{1}{n} \sum_{i=1}^n x_i$$

MLE for Fitting Scalar Gaussian distribution (cont.)



- The plot in blue color is the histogram of input data.
- python function for plotting the histogram: `matplotlib.pyplot.hist()`

Sampling from a Scalar Gaussian Distribution

Prior Knowledge:

- Let $Y = \mu^* + \sigma^{2,*} X$, where $X \sim \mathcal{N}(x|0, 1)$
- It can be shown that $Y \sim \mathcal{N}(\mu^*, \sigma^{2,*})$.

Sampling procedure:

- Generate samples for X , denoted as $\{x_i\}_{i=1}^n$
- Compute $y_i = \mu^* + \sigma^{2,*} x_i$ to obtain the new data samples $\{y_i\}_{i=1}^n$.

Python functions for generating $\{x_i\}_{i=1}^n$:

- Pytorch function: `torch.randn()`
- numpy function: `numpy.random.randn()`

MLE for Fitting multi-dimensional Gaussian distribution

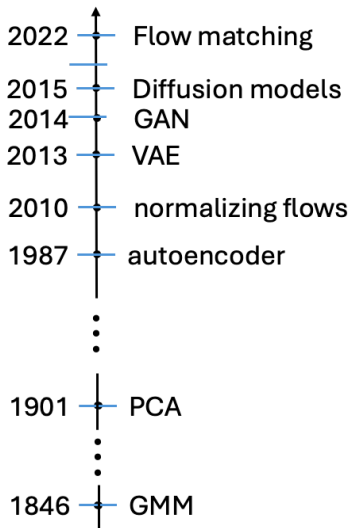
Brief summary:

- Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
- Let $p_{\boldsymbol{\theta}}(\cdot)$ be the Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}^2)$ where $\boldsymbol{\theta} = \{\boldsymbol{\mu}, \boldsymbol{\Sigma}\}$.
- Fitting the data by maximizing

$$\begin{aligned}\boldsymbol{\mu}^*, \boldsymbol{\Sigma}^* &= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \log p_{\boldsymbol{\theta}}(\mathbf{x}_i) \\&= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \log \frac{1}{|\boldsymbol{\Sigma}|^{1/2} (2\pi)^{d/2}} e^{-\frac{1}{2}(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu})} \\&= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \left(-\frac{1}{2}(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu}) - \frac{1}{2} \log |\boldsymbol{\Sigma}| \right) \\&= \arg \min_{\boldsymbol{\theta}} \underbrace{\frac{1}{n} \sum_{i=1}^n \left(\frac{1}{2}(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu}) + \frac{1}{2} \log |\boldsymbol{\Sigma}| \right)}_{L(\boldsymbol{\mu}, \boldsymbol{\Sigma})}\end{aligned}$$

Gaussian Mixture Model (GMM)

GMM was invented back in 1846. It is still widely used nowadays in machine learning field.



Gaussian Mixture Model (GMM)

- A Gaussian Mixture Model (GMM) is a probabilistic generative model (PGM) that assumes a dataset's underlying distribution is a weighted sum of multiple Gaussian (normal) distributions.
- Its PDF $p_{\theta}(\mathbf{x})$ takes the form of

$$p_{\theta}(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- K is the number of components in GMM.
- π_k is the weight for the k th component, which tells us how much that particular Gaussian contributes to the overall model. These weights must satisfy

$$\{\pi_k \geq 0\}_{k=1}^K, \text{ and } \sum_{k=1}^K \pi_k = 1$$

- $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ is the Gaussian distribution for the k th component.
- The parameter θ includes $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$.

Gaussian Mixture Model (cont.)

- Consider a GMM model with $p_{\theta}(\mathbf{x})$

$$p_{\theta}(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- We define a set of discrete latent random variables $\mathbf{Z} = \{Z_1, Z_2, \dots, Z_K\}$
 - Each variable $Z_k \in \{0, 1\}$ and $P_{Z_k}(z_k = 1) = \pi_k$
 - $\sum_k Z_k = 1$ and $\sum_{k=1}^K \pi_k = 1$
 - The latent variables in $\mathbf{Z}$ are mutually dependent on each other.

observable variable latent variable



Gaussian Mixture Model (cont.)

- Consider a GMM model with $p_{\theta}(\mathbf{x})$

$$p_{\theta}(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- We define a set of discrete latent random variables $\mathbf{Z} = \{Z_1, Z_2, \dots, Z_K\}$
 - Each variable $Z_k \in \{0, 1\}$ and $P_{Z_k}(z_k = 1) = \pi_k$
 - $\sum_k Z_k = 1$ and $\sum_{k=1}^K \pi_k = 1$
 - The latent variables in $\mathbf{Z}$ are mutually dependent on each other.
- $p(\mathbf{x}, \mathbf{z}) = P_{\mathbf{Z}}(\mathbf{z})p(\mathbf{x}|\mathbf{z})$
- $P_{\mathbf{Z}}(\mathbf{z}) = \prod_{k=1}^K \pi_k^{z_k}$
- $p(\mathbf{x}|\mathbf{z}) = \prod_{k=1}^K \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k}$

Remark: $p_{\theta}(\mathbf{x})$ is the marginal distribution of $p(\mathbf{x}, \mathbf{z})$.

Gaussian Mixture Model (cont.)

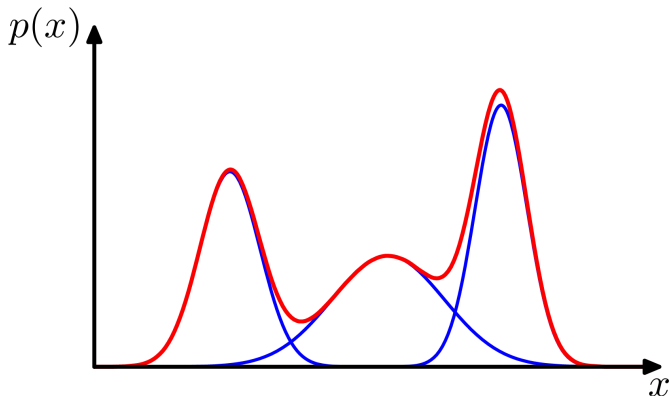
Since $p(\mathbf{x}, \mathbf{z}) = P_{\mathbf{Z}}(\mathbf{z})p(\mathbf{x}|\mathbf{z}) = p(\mathbf{x})p(\mathbf{z}|\mathbf{x})$

- The posterior distribution can be computed as $\gamma(z_k) = p(z_k = 1|\mathbf{x})$:

$$\begin{aligned}\gamma(z_k) &= p(z_k = 1|\mathbf{x}) \\ &= \frac{p(z_k = 1)p(\mathbf{x}|z_k = 1)}{\sum_{j=1}^K p(z_j = 1)p(\mathbf{x}|z_j = 1)} \\ &= \frac{\pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}\end{aligned}$$

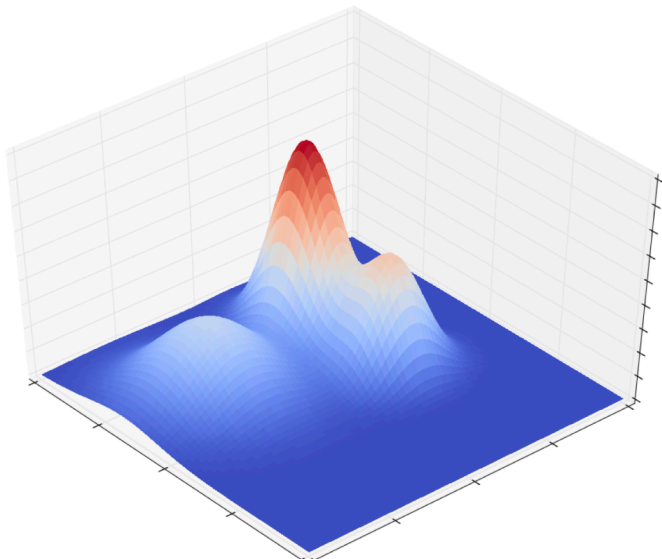
Gaussian Mixture Model (GMM) (cont.)

- In the example below, the GMM model has three components.
- Each component is a scalar Gaussian distribution



Gaussian Mixture Model (GMM) (cont.)

- In the example below, the GMM model has three components.
- Each component is a 2d Gaussian distribution.



Sampling from a GMM model

- First draw a sample z from the latent random vector $\mathbf{Z}$ such that $z_j \in \{0, 1\}$ and $\sum_{j=1}^K z_j = 1$.
- Note that only one Gaussian component will be activated. Suppose $z_j = 1$, meaning that the j th component is activated.
- Draw a sample from j th Gaussian distribution, $\mathbf{x} \sim \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$.

Remark: Sampling for a GMM is straightforward.

Training a GMM with Maximum Likelihood (Not optimal)

- A training dataset: $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$
- The likelihood function of the n data samples is

$$\prod_{i=1}^n p_{\theta}(\mathbf{x}_i) = \prod_{i=1}^n \left(\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right)$$

- The log-likelihood function is

$$\begin{aligned} L(\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K) &= \log \prod_{i=1}^n p_{\theta}(\mathbf{x}_i) \\ &= \sum_{i=1}^n \log \left(\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right) \end{aligned}$$

- In principle, we can estimate $\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K$ by computing the gradient of the log-likelihood with respect to these parameters and using gradient-based optimization (**not optimal**).

Training a GMM by Expectation-Maximization (EM)

Principles of the EM method:

- EM derives a lower bound B function on the likelihood L ($B \leq L$).
- Instead of maximizing L directly, EM maximizes the lower bound function B .
-

Training a GMM by Expectation-Maximization (EM)

Principles of the EM method:

- EM derives a lower bound B function on the likelihood L ($B \leq L$).
- Instead of maximizing L directly, EM maximizes the lower bound function B .
- Question: How to determine B ? Using Jensen's inequality

$$\log \mathbb{E}_{p_Z}[g(z)] \geq \mathbb{E}_{p_Z}[\log(g(z))]$$

Derivation of Lower Bound B

Prior knowledge:

- Each data sample $\mathbf{x}_i$ has a corresponding latent sample $\mathbf{z}_i = \{z_{i,1}, z_{i,2}, \dots, z_{i,K}\}$.

The L function can be equivalently represented as

$$\begin{aligned} & L(\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K) \\ &= \sum_{i=1}^n \log \left(\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right) \\ &\stackrel{(a)}{=} \sum_{i=1}^n \log \left(\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \frac{\gamma_{ij}}{\gamma_{ij}} \right) \quad [\text{posterior distribution: } \gamma_{ij} = p(z_{i,j} = 1 | \mathbf{x}_i)] \\ &\stackrel{(b)}{=} \sum_{i=1}^n \log \sum_{j=1}^K \gamma_{ij} \frac{\pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\gamma_{ij}} \\ &\stackrel{(c)}{=} \sum_{i=1}^n \log \mathbb{E}_{\mathbf{z}_i} \frac{\pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\gamma_{ij}} \end{aligned}$$

Derivation of Lower Bound B (cont.)

The L function can be lower bounded by:

$$\begin{aligned} L(\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K) &= \sum_{i=1}^n \log \mathbb{E}_{\mathbf{Z}_i} \frac{\pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\gamma_{ij}} \\ &\geq \sum_{i=1}^n \mathbb{E}_{\mathbf{Z}_i} \log \frac{\pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\gamma_{ij}} \quad \left[\text{applying } \log \mathbb{E}_{p_Z}[g(\mathbf{z})] \geq \mathbb{E}_{p_Z}[\log(g(\mathbf{z}))] \right] \\ &= B(\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K, \{\gamma_{ij}\}) \end{aligned}$$

The B function takes the following form:

$$\begin{aligned} B(\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K, \{\gamma_{ij}\}) &= \sum_{i=1}^n \sum_{j=1}^K \gamma_{ij} \log \pi_j + \sum_{i=1}^n \sum_{j=1}^K \gamma_{ij} \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) - \sum_{i=1}^n \sum_{j=1}^K \gamma_{ij} \log \gamma_{ij} \end{aligned}$$

Update procedure of EM for GMM

- EM is an iterative process, maximizing the lower bound B until convergence.
- At each iteration, we perform alternative optimization:
 - Fixing $\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K$, maximize B over $\{\gamma_{ij} | i = 1, \dots, n, j = 1, \dots, K\}$.
 - Fixing $\{\gamma_{ij} | i = 1, \dots, n, j = 1, \dots, K\}$, maximize B over $\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K$.

Optimization with constraints

$$\begin{aligned} & B(\{\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K, \{\gamma_{ij}\}) \\ &= \sum_{i=1}^n \sum_{j=1}^K \gamma_{ij} \log \pi_j + \sum_{i=1}^n \sum_{j=1}^K \gamma_{ij} \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) - \sum_{i=1}^n \sum_{j=1}^K \gamma_{ij} \log \gamma_{ij}, \end{aligned}$$

$$\text{subject to } \sum_{j=1}^K \pi_j = 1 \text{ and } \sum_{j=1}^K \gamma_{ij} = 1 \quad \forall i \in \{1, \dots, n\}$$

Update procedure of EM for GMM

E-Step

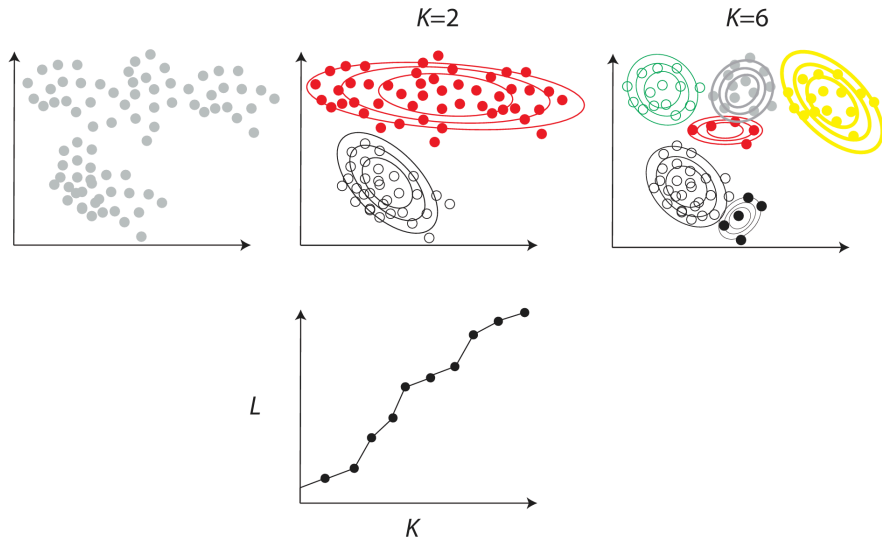
$$\gamma_{ij} = \frac{\pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}$$
$$\forall i = 1, \dots, n, \forall j = 1, \dots, K$$

M-Step

$$\pi_j = \frac{1}{n} \sum_{i=1}^n \gamma_{ij}$$
$$\boldsymbol{\mu}_j = \frac{\sum_{i=1}^n \gamma_{ij} \mathbf{x}_i}{\sum_{i=1}^n \gamma_{ij}}$$
$$\boldsymbol{\Sigma}_j = \frac{\sum_{i=1}^n \gamma_{ij} (\mathbf{x}_i - \boldsymbol{\mu}_j)(\mathbf{x}_i - \boldsymbol{\mu}_j)^T}{\sum_{i=1}^n \gamma_{ij}}$$
$$\forall j = 1, \dots, K$$

- The E-step calculates the probability each data point belongs to each Gaussian.
- The M-step updates the parameters of the Gaussians based on the probabilities $\{\gamma_{ij}\}$

Impact of components K for GMMs



Remark: As K increases, the log-likelihood of the associated trained GMM would also increase.

Overfitting with GMM

Definition

Overfitting is a machine learning error that occurs when a model learns its training data "too well," memorizing specific details and noise instead of the underlying patterns. This leads to a model that performs exceptionally on the training data but fails to generalize to and accurately predict new, unseen data, making it unreliable in real-world applications.

- As K increases in GMM, overfitting might occur. That is, GMM is trained too well on the training data, and performs poorly on the test data.
- Detection of overfitting:
 - Compute the average log-likelihood of the test data. If the average log-likelihood of the test data is far smaller than that of the training data, overfitting may occur.

Summary

PGMs	GMMs	VAEs	GANs	NFs	Diffusion Models
Quality	poor for high dimension				
Sampling	Easy				
Evaluating $p_{\theta}(\mathbf{x})$	Easy				

References

The slides regarding GMM heavily rely on the following lecture note:

- Lecture notes from Kia Nazarpour

Questions?